

8.2.2 Random Forests

Random forests cải thiện bagged trees bằng một small tweak nhằm decorrelate các trees. Giống bagging, ta build nhiều decision trees trên bootstrapped training samples. Tuy nhiên, khi build các decision trees này, mỗi lần xét một split trong tree, một random sample gồm m predictors được chọn làm split candidates từ full set gồm p predictors. Split chỉ được phép dùng một trong m predictors đó. Một fresh sample gồm m predictors được lấy tại mỗi split, và thông thường chọn $m \approx \sqrt{p}$ —tức number of predictors được xét tại mỗi split xấp xỉ square root của tổng number of predictors (4 trong số 13 đối với data **Heart**).

random forest

Nói cách khác, khi build random forest, tại mỗi split algorithm thậm chí không được phép xét phần lớn available predictors. Điều này nghe có vẻ lạ nhưng có rationale rất clever. Giả sử data set có một predictor rất strong cùng một số predictors moderately strong. Trong collection of bagged trees, phần lớn hoặc tất cả trees sẽ dùng strong predictor này ở top split. Kết quả là các bagged trees trông rất giống nhau và predictions từ chúng sẽ highly correlated.

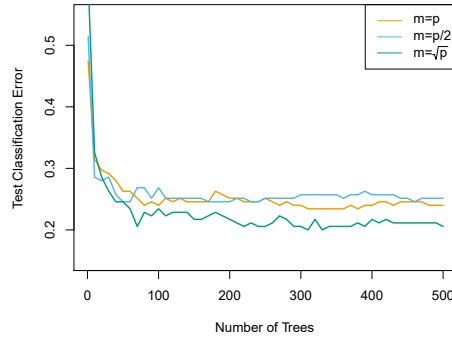
Averaging nhiều highly correlated quantities không giảm variance nhiều như averaging nhiều uncorrelated quantities. Do đó trong setting này, bagging có thể không giảm variance đáng kể so với single tree. Random forests giải quyết vấn đề bằng cách buộc mỗi split chỉ xét một subset predictors. Trung bình $(p-m)/p$ splits thậm chí không xét strong predictor, nên các predictors khác có thêm cơ hội được chọn. Có thể xem process này là decorrelating the trees, làm average của resulting trees ít variable hơn và reliable hơn.

Main difference giữa bagging và random forests là predictor subset size m . Nếu random forest dùng $m = p$ thì chính là bagging. Trên data **Heart**, random forests với $m = \sqrt{p}$ giảm cả test error lẫn OOB error so với bagging (Figure 8.8).

Small m thường hữu ích khi có nhiều correlated predictors. Random forests được apply lên high-dimensional biological data set gồm expression measurements của 4,718 genes trên tissue samples từ 349 patients. Con người có khoảng 20,000 genes, và từng gene có levels of activity hay expression khác nhau trong particular cells, tissues và biological conditions. Mỗi patient sample trong data này có qualitative label với 15 levels: normal hoặc 1 trong 14 cancer types. Goal là predict cancer type bằng random forests dựa trên 500 genes có largest variance trong training set.

Ta random divide observations thành training và test set, rồi apply random forests lên training set với ba values khác nhau của number of splitting variables m . Results ở Figure 8.10. Error rate của single tree là 45.7%, null rate là 75.4%.⁴ Ta thấy 400 trees là đủ cho good performance và choice $m = \sqrt{p}$ cải thiện test error nhẹ so với bagging ($m = p$). Giống bagging, random forests không overfit khi tăng B , nên trong practice ta chọn B đủ lớn để error rate đã settle down.

⁴Null rate có được khi đơn giản classify mọi observation vào dominant class overall, trong case này là normal class.



HÌNH 8.10. Results từ random forests cho 15-class gene expression data set với $p = 500$ predictors. Test error được display theo number of trees. Mỗi colored line ứng với một value khác nhau của m , number of predictors available for splitting tại mỗi interior tree node. Random forests ($m < p$) cho cải thiện nhẹ so với bagging ($m = p$). Một single classification tree có error rate 45.7%.

8.2.3 Boosting

Ta tiếp tục với boosting, một approach khác để improve predictions từ decision tree. Giống bagging, boosting là general approach có thể apply cho nhiều statistical learning methods trong regression hoặc classification. Ở đây ta giới hạn discussion vào decision trees.

boosting

Bagging tạo multiple copies của original training data set bằng bootstrap, fit một separate decision tree trên mỗi copy, rồi combine tất cả trees thành single predictive model. Mỗi tree được build trên bootstrap data set độc lập với các trees khác. Boosting tương tự nhưng trees được grow sequentially: mỗi tree được grow dùng information từ previously grown trees. Boosting không dùng bootstrap sampling; mỗi tree được fit trên modified version của original data set.

Trước hết xét regression. Giống bagging, boosting combine một large number of decision trees $\hat{f}^1, \dots, \hat{f}^B$. Procedure được mô tả trong Algorithm 8.2. Khác với fit một single large decision tree—fit data rất hard và có thể overfit—boosting học chậm. Với current model, ta fit decision tree lên residuals của model, tức dùng current residuals thay vì outcome Y làm response. Sau đó add new tree vào fitted function để update residuals.

Mỗi tree có thể khá small, chỉ vài terminal nodes, controlled bởi parameter d . Bằng cách fit small trees vào residuals, ta từ từ improve $\hat{f}$ ở nơi nó chưa perform tốt. Shrinkage parameter λ làm process còn chậm hơn, cho phép nhiều trees có shapes khác nhau cùng attack residuals. Nhìn chung statistical learning approaches học chậm thường perform tốt. Trong boosting, trái với bagging, construction của mỗi tree phụ thuộc strongly vào trees đã được grow trước đó. Boosting classification trees tương tự nhưng phức tạp hơn một chút; details được omitted.

Algorithm 8.2 Boosting for Regression Trees

1. Set $\hat{f}(x) = 0$ và $r_i = y_i$ cho mọi i trong training set.
2. Với $b = 1, 2, \dots, B$, repeat:

- (a) Fit một tree $\hat{f}^b$ với d splits ($d + 1$ terminal nodes) vào training data (X, r) .
- (b) Update $\hat{f}$ bằng cách add một shrunken version của new tree:

$$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^b(x). \quad (8.10)$$

- (c) Update residuals,

$$r_i \leftarrow r_i - \lambda \hat{f}^b(x_i). \quad (8.11)$$

- 3. Output boosted model,

$$\hat{f}(x) = \sum_{b=1}^B \lambda \hat{f}^b(x). \quad (8.12)$$

Boosting có ba tuning parameters:

1. Number of trees B . Không giống bagging và random forests, boosting có thể overfit nếu B quá lớn, dù overfitting thường diễn ra chậm hoặc thậm chí không xảy ra. Ta dùng cross-validation để select B .
2. Shrinkage parameter λ , một small positive number controlling rate at which boosting learns. Typical values là 0.01 hoặc 0.001, và choice phù hợp phụ thuộc problem. Very small λ có thể yêu cầu very large B để đạt good performance.
3. Number d of splits trong mỗi tree, controlling complexity của boosted ensemble. Thường $d = 1$ works well; mỗi tree khi đó là một *stump*, gồm single split. Trong case này boosted ensemble fit một additive model vì mỗi term chỉ involve một variable. Tổng quát hơn d là *interaction depth*, controlling interaction order của boosted model, vì d splits involve nhiều nhất d variables.

*stump; interaction
depth*

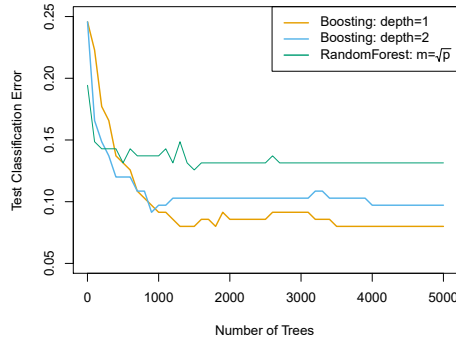
Trong Figure 8.11, boosting được apply lên 15-class cancer gene expression data set để develop classifier phân biệt normal class với 14 cancer classes. Test error được display theo total number of trees và interaction depth d . Simple stumps với depth one perform tốt nếu include đủ nhiều. Model này outperform depth-two model, và cả hai outperform random forest. Điều này highlight một difference giữa boosting và random forests: vì growth của particular tree trong boosting có xét tới các trees đã grow, smaller trees thường sufficient. Smaller trees cũng giúp interpretability; chẳng hạn stumps dẫn tới additive model.

8.2.4 Bayesian Additive Regression Trees

Cuối cùng ta discuss Bayesian additive regression trees (BART), một ensemble method khác dùng decision trees làm building blocks. Để đơn giản, ta trình bày BART cho regression thay vì classification.

*Bayesian additive
regression trees*

Bagging và random forests predict bằng average regression trees, mỗi tree được build từ random sample of data và/hoặc predictors; each tree built separately. Boosting dùng weighted sum of trees, mỗi tree fit vào residual của current fit, nên mỗi new tree cố capture signal chưa được current set of



HÌNH 8.11. Results từ boosting và random forests trên 15-class gene expression data set để predict cancer versus normal. Test error được display theo number of trees. Với hai boosted models, $\lambda = 0.01$. Depth-1 trees hơi outperform depth-2 trees và cả hai outperform random forest, dù standard errors khoảng 0.02 nên none of these differences significant. Test error rate của single tree là 24%.

trees account for. BART liên quan cả hai: mỗi tree constructed randomly như bagging/random forests và cũng cố capture signal chưa được current model account for như boosting. Main novelty của BART là cách generate new trees.

Ký hiệu K là number of regression trees và B là number of iterations BART chạy. $\hat{f}_k^b(x)$ là prediction tại x của k th regression tree trong b th iteration. Cuối mỗi iteration, K trees được sum:

$$\hat{f}^b(x) = \sum_{k=1}^K \hat{f}_k^b(x), \quad b = 1, \dots, B.$$

Trong first iteration, all trees initialized với single root node và

$$\hat{f}_k^1(x) = \frac{1}{nK} \sum_{i=1}^n y_i,$$

mean response divided by total number of trees. Vì vậy $\hat{f}^1(x) = \sum_{k=1}^K \hat{f}_k^1(x) = n^{-1} \sum_i y_i$.

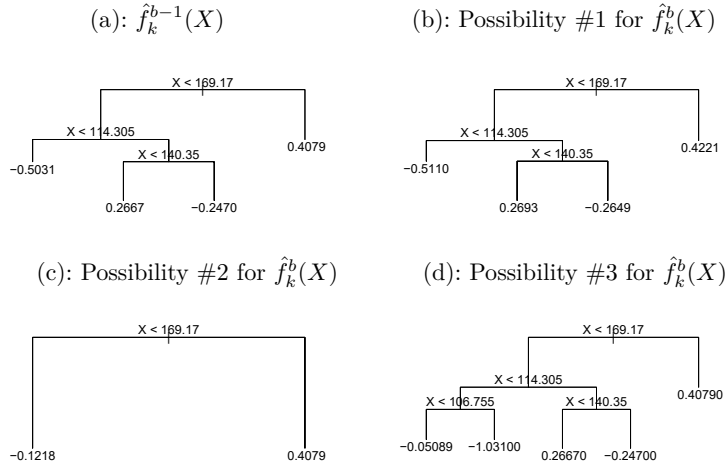
Ở subsequent iterations, BART update từng tree trong K trees, one at a time. Trong b th iteration, để update k th tree, ta subtract từ mỗi response value predictions của all trees trừ k th tree, tạo partial residual

$$r_i = y_i - \sum_{k' < k} \hat{f}_{k'}^b(x_i) - \sum_{k' > k} \hat{f}_{k'}^{b-1}(x_i), \quad i = 1, \dots, n.$$

Thay vì fit fresh tree vào partial residual này, BART randomly choose một perturbation của tree từ previous iteration $\hat{f}_k^{b-1}$ từ set of possible perturbations, favoring perturbations improve fit. Perturbation có hai components:

1. Có thể change tree structure bằng adding hoặc pruning branches.
2. Có thể change prediction trong mỗi terminal node.

Figure 8.12 minh họa possible perturbations.



HÌNH 8.12. Schematic của perturbed trees từ BART algorithm. (a): k th tree ở $(b-1)$ st iteration, $\hat{f}_k^{b-1}(X)$. Panels (b)–(d) show ba trong nhiều possibilities cho $\hat{f}_k^b(X)$ given form của $\hat{f}_k^{b-1}(X)$. (b): Có thể giữ same tree structure nhưng terminal-node predictions khác. (c): Có thể prune $\hat{f}_k^{b-1}(X)$. (d): Hoặc $\hat{f}_k^b(X)$ có nhiều terminal nodes hơn.

Output của BART là collection of prediction models

$$\hat{f}^b(x) = \sum_{k=1}^K \hat{f}_k^b(x), \quad b = 1, 2, \dots, B.$$

Algorithm 8.3 Bayesian Additive Regression Trees

1. Let $\hat{f}_1^1(x) = \hat{f}_2^1(x) = \dots = \hat{f}_K^1(x) = \frac{1}{nK} \sum_{i=1}^n y_i$.

2. Compute $\hat{f}^1(x) = \sum_{k=1}^K \hat{f}_k^1(x) = \frac{1}{n} \sum_{i=1}^n y_i$.

3. Với $b = 2, \dots, B$:

(a) Với $k = 1, 2, \dots, K$:

i. Với $i = 1, \dots, n$, compute current partial residual

$$r_i = y_i - \sum_{k' < k} \hat{f}_{k'}^b(x_i) - \sum_{k' > k} \hat{f}_{k'}^{b-1}(x_i).$$

ii. Fit new tree $\hat{f}_k^b(x)$ vào r_i bằng randomly perturbing k th tree từ previous iteration $\hat{f}_k^{b-1}(x)$. Perturbations improve fit được favored.

(b) Compute $\hat{f}^b(x) = \sum_{k=1}^K \hat{f}_k^b(x)$.

4. Compute mean sau L burn-in samples,

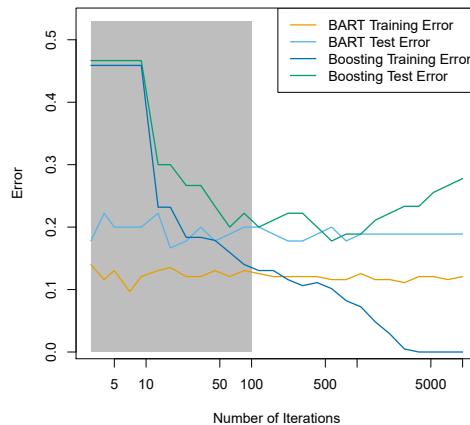
$$\hat{f}(x) = \frac{1}{B-L} \sum_{b=L+1}^B \hat{f}^b(x).$$

Ta thường discard vài prediction models đầu vì models ở early iterations—gọi là *burn-in period*—thường chưa give good results. Gọi L là number of burn-in iterations, chẳng hạn $L = 200$. Single prediction được lấy bằng average after burn-in. Nhưng cũng có thể compute quantities khác average: percentiles của $\hat{f}^{L+1}(x), \dots, \hat{f}^B(x)$ provide measure of uncertainty trong final prediction.

burn-in

Key element của BART là trong Step 3(a)ii ta không fit fresh tree vào current partial residual. Thay vào đó, cố improve fit bằng slightly modifying tree ở previous iteration (Figure 8.12). Roughly speaking, điều này guards against overfitting vì giới hạn mức độ fit data “hard” ở mỗi iteration. Individual trees cũng thường small; limiting tree size giúp tránh overfitting.

Figure 8.13 show BART trên data **Heart** với $K = 200$ trees, khi number of iterations tăng đến 10,000. Trong initial iterations, test và training errors jump around. Sau burn-in period, error rates settle down. Chỉ có small difference giữa training và test error, cho thấy tree perturbation process phần lớn avoid overfitting.



HÌNH 8.13. BART và boosting results cho data **Heart**. Cả training và test errors đều được display. Sau burn-in period 100 iterations (shown in gray), BART error rates settle down. Boosting bắt đầu overfit sau vài hundred iterations.

Training và test errors cho boosting cũng ở Figure 8.13. Test error của boosting approaches BART nhưng sau đó tăng khi number of iterations tăng. Training error của boosting giảm khi iterations tăng, cho thấy boosting đã overfit data.

Dù details ngoài scope, BART có thể được xem như Bayesian approach để fit ensemble of trees: mỗi lần randomly perturb tree để fit residuals, ta thực chất draw một new tree từ posterior distribution. Bayesian connection này là motivation cho tên BART. Algorithm 8.3 cũng có thể xem như một Markov chain Monte Carlo algorithm để fit BART model.

*Markov chain
Monte Carlo*

Khi apply BART, phải select number of trees K , number of iterations B , và number of burn-in iterations L . Thường chọn large B và K , moderate L ; chẳng hạn $K = 200$, $B = 1,000$, $L = 100$ là reasonable choice. BART có impressive out-of-box performance—perform well với minimal tuning.

8.2.5 Summary of Tree Ensemble Methods

Trees là attractive weak learners cho ensemble method nhờ flexibility và khả năng handle predictors of mixed types (qualitative lẫn quantitative). Ta đã thấy bốn approaches để fit ensemble of trees: bagging, random forests, boosting, và BART.

- Trong bagging, trees grow independently trên random samples of observations. Vì thế trees khá similar; bagging có thể mắc vào local optima và fail to thoroughly explore model space.
- Trong random forests, trees cũng grow independently trên random samples of observations. Nhưng mỗi split dùng random subset of features, decorrelating trees và dẫn tới more thorough exploration of model space so với bagging.
- Trong boosting, chỉ dùng original data, không draw random samples. Trees grow successively theo “slow” learning approach: mỗi new tree fit vào signal còn lại từ earlier trees rồi được shrunken trước khi dùng.
- Trong BART, cũng chỉ dùng original data và grow trees successively. Tuy nhiên, mỗi tree được perturbed để avoid local minima và explore model space thoroughly hơn.

8.3 Lab: Tree-Based Methods

Ta import một số usual libraries ở top level.

```
In [1]: import numpy as np
import pandas as pd
from matplotlib.pyplot import subplots
from statsmodels.datasets import get_rdataset
import sklearn.model_selection as skm
from ISLP import load_data, confusion_table
from ISLP.models import ModelSpec as MS
```

Ta cũng collect các new imports cần cho lab này.

```
In [2]: from sklearn.tree import (DecisionTreeClassifier as DTC,
                             DecisionTreeRegressor as DTR,
                             plot_tree,
                             export_text)
from sklearn.metrics import (accuracy_score,
                             log_loss)
from sklearn.ensemble import \
    (RandomForestRegressor as RF,
     GradientBoostingRegressor as GBR)
from ISLP.bart import BART
```

8.3.1 Fitting Classification Trees

Trước hết ta dùng classification trees để analyze data set **Carseats**. Trong data này, **Sales** là continuous variable, nên ta recode thành binary variable. Function **where()** tạo variable **High**, bằng **Yes** nếu **Sales**>8 và **No** otherwise.

where()

```
In [3]: Carseats = load_data('Carseats')
        High = np.where(Carseats.Sales > 8,
                        "Yes",
                        "No")
```

Ta dùng `DecisionTreeClassifier()` để fit classification tree predict High bằng tất cả variables trừ Sales. Trước đó phải form model matrix như khi fit regression models.

DTC

```
In [4]: model = MS(Carseats.columns.drop('Sales'), intercept=False)
        D = model.fit_transform(Carseats)
        feature_names = list(D.columns)
        X = np.asarray(D)
```

Ta convert D từ data frame thành array X, cần cho một số analysis bên dưới. `feature_names` sẽ được dùng annotate plots. Classifier có nhiều options như `max_depth` (tree sâu đến đâu), `min_samples_split` (minimum observations để node eligible for splitting), và `criterion` (Gini hay cross-entropy). Ta cũng set `random_state` để reproducibility; ties trong split criterion được break randomly.

```
In [5]: clf = DTC(criterion='entropy',
                  max_depth=3,
                  random_state=0)
        clf.fit(X, High)
```

```
Out[5]: DecisionTreeClassifier(criterion='entropy', max_depth=3)
```

Trong discussion về qualitative features ở Section 3.3, một linear regression model có thể represent feature như vậy bằng matrix of dummy variables (one-hot-encoding) trong model matrix. Với decision tree có cách natural hơn: mỗi split partition levels thành hai groups. Tuy nhiên sklearn implementation không tận dụng approach này; nó treats one-hot-encoded levels như separate variables.

```
In [6]: accuracy_score(High, clf.predict(X))
```

```
Out[6]: 0.7275
```

Với classification trees, deviance có thể access bằng `log_loss()`. Theo notation của chapter, quantity liên quan là

$$-2 \sum_m \sum_k n_{mk} \log \hat{p}_{mk},$$

trong đó n_{mk} là number of observations ở m th terminal node thuộc k th class.

`log_loss()`

```
In [7]: resid_dev = np.sum(log_loss(High, clf.predict_proba(X)))
        resid_dev
```

```
Out[7]: 0.4711
```

Quantity này closely related to entropy trong (8.7). Small deviance indi-

cates tree fit training data tốt. Một attractive property của trees là display graphically. Ở đây dùng `plot_tree()` để display tree structure (không show trong sách).

In [8]:

```
ax = subplots(figsize=(12,12))[1]
plot_tree(clf,
          feature_names=feature_names,
          ax=ax);
```

Important indicator nhất của Sales dường như là **ShelveLoc**. Ta xem text representation bằng `export_text()`, nó display split criterion cho mỗi branch và overall prediction cho leaf nodes. Với `show_weights=True`, cũng thấy number of observations trong leaf có values **Yes** và **No**.

`export_text()`

In [9]:

```
print(export_text(clf,
                  feature_names=feature_names,
                  show_weights=True))
```

Out[9]:

```
|--- ShelveLoc[Good] <= 0.50
|   |--- Price <= 92.50
|   |   |--- Income <= 57.00
|   |   |   |--- weights: [7.00, 3.00] class: No
|   |   |   |--- Income > 57.00
|   |   |   |--- weights: [7.00, 29.00] class: Yes
|   |   |--- Price > 92.50
|   |   |--- Advertising <= 13.50
|   |   |   |--- weights: [183.00, 41.00] class: No
|   |   |   |--- Advertising > 13.50
|   |   |   |--- weights: [20.00, 25.00] class: Yes
|--- ShelveLoc[Good] > 0.50
|   |--- Price <= 135.00
|   |   |--- US[Yes] <= 0.50
|   |   |   |--- weights: [6.00, 11.00] class: Yes
|   |   |   |--- US[Yes] > 0.50
|   |   |   |--- weights: [2.00, 49.00] class: Yes
|   |   |--- Price > 135.00
|   |   |--- Income <= 46.00
|   |   |   |--- weights: [6.00, 0.00] class: No
|   |   |   |--- Income > 46.00
|   |   |   |--- weights: [5.00, 6.00] class: Yes
```

Để properly evaluate classification-tree performance, phải estimate test error thay vì chỉ compute training error. Ta split observations thành training và test set, build tree trên training set rồi evaluate trên test data. Pattern giống Chapter 6, chỉ thay linear models bằng decision trees; validation code gần như identical. Approach này predict correct 68.5% locations trong test set.

In [10]:

```
validation = skm.ShuffleSplit(n_splits=1,
                              test_size=200,
                              random_state=0)

results = skm.cross_validate(clf,
                              D,
                              High,
                              cv=validation)

results['test_score']
```

Out[10]:

```
array([0.685])
```

Tiếp theo xét pruning có improve classification performance không. Ta split data thành training/test set, use cross-validation prune tree trên training set, rồi evaluate pruned tree trên test set.

```
In [11]: (X_train,
          X_test,
          High_train,
          High_test) = skm.train_test_split(X,
                                             High,
                                             test_size=0.5,
                                             random_state=0)
```

Refit full tree trên training set; không set `max_depth` vì sẽ learn qua cross-validation.

```
In [12]: clf = DTC(criterion='entropy', random_state=0)
          clf.fit(X_train, High_train)
          accuracy_score(High_test, clf.predict(X_test))
```

Out[12]: 0.735

Method `cost_complexity_pruning_path()` của `clf` extract cost-complexity values.

```
In [13]: ccp_path = clf.cost_complexity_pruning_path(X_train, High_train)
          kfold = skm.KFold(10,
                             random_state=1,
                             shuffle=True)
```

*cost-complexity
path*

Set of impurities và α values này cho phép extract optimal one bằng cross-validation.

```
In [14]: grid = skm.GridSearchCV(clf,
                                  {'ccp_alpha': ccp_path.ccp_alphas},
                                  refit=True,
                                  cv=kfold,
                                  scoring='accuracy')
          grid.fit(X_train, High_train)
          grid.best_score_
```

Out[14]: 0.685

Ta xem pruned tree.

```
In [15]: ax = subplots(figsize=(12, 12))[1]
          best_ = grid.best_estimator_
          plot_tree(best_,
                    feature_names=feature_names,
                    ax=ax);
```

Tree khá bushy. Có thể count leaves hoặc query `best_`.

```
In [16]: best_.tree_.n_leaves
```

Out[16]: 30

Tree với 30 terminal nodes cho lowest cross-validation error rate, accuracy

68.5%. Để đánh giá trên test data, lại apply `predict()`.

In [17]:

```
print(accuracy_score(High_test,
                     best_.predict(X_test)))
confusion = confusion_table(best_.predict(X_test),
                           High_test)

confusion
```

Out[17]:

0.72

Truth	No	Yes
Predicted		
No	108	61
Yes	10	21

Bây giờ 72.0% test observations classified correctly, slightly worse than full tree (35 leaves). Cross-validation không giúp nhiều ở đây: chỉ prune 5 leaves với cost là error hơi worse. Results sẽ change nếu đổi random seeds; dù cross-validation là unbiased approach cho model selection, nó vẫn có variance.

8.3.2 Fitting Regression Trees

Ta fit regression tree lên data Boston; steps tương tự classification trees.

In [18]:

```
Boston = load_data("Boston")
model = MS(Boston.columns.drop('medv'), intercept=False)
D = model.fit_transform(Boston)
feature_names = list(D.columns)
X = np.asarray(D)
```

Đầu tiên split data thành training/test sets, dùng 30% data cho test set.

In [19]:

```
(X_train,
 X_test,
 y_train,
 y_test) = skm.train_test_split(X,
                                Boston['medv'],
                                test_size=0.3,
                                random_state=0)
```

Sau khi form training/test data sets, fit regression tree.

In [20]:

```
reg = DTR(max_depth=3)
reg.fit(X_train, y_train)
ax = subplots(figsize=(12,12))[1]
plot_tree(reg,
          feature_names=feature_names,
          ax=ax);
```

Variable `lstat` measures percentage of individuals có lower socioeconomic status. Tree indicates lower `lstat` corresponds to more expensive houses. Tree predicts median house price \$12,042 cho small homes (`rm`<6.8) trong suburbs nơi residents có low socioeconomic status (`lstat`>14.4) và crime rate moderate (`crim`>5.8).

Ta dùng cross-validation để xem pruning có improve performance không.

```
In [21]: ccp_path = reg.cost_complexity_pruning_path(X_train, y_train)
kfold = skm.KFold(5,
                  shuffle=True,
                  random_state=10)
grid = skm.GridSearchCV(reg,
                        {'ccp_alpha': ccp_path.ccp_alphas},
                        refit=True,
                        cv=kfold,
                        scoring='neg_mean_squared_error')
G = grid.fit(X_train, y_train)
```

Theo cross-validation results, dùng pruned tree để predict test set.

```
In [22]: best_ = grid.best_estimator_
np.mean((y_test - best_.predict(X_test))**2)
```

Out[22]: 28.07

Test-set MSE là 28.07. Square root của MSE khoảng 5.30, nghĩa là test predictions cách true median home value khoảng \$5,300. Ta plot best tree để xem interpretability.

```
In [23]: ax = subplots(figsize=(12,12))[1]
plot_tree(G.best_estimator_,
          feature_names=feature_names,
          ax=ax);
```

8.3.3 Bagging and Random Forests

Ta apply bagging và random forests lên data Boston bằng **RandomForestRegressor()** từ package **sklearn.ensemble**. Bagging là special case của random forest với $m = p$, nên function này perform được cả hai. Bắt đầu với bagging.

*random forest
regressor*

```
In [24]: bag_boston = RF(max_features=X_train.shape[1], random_state=0)
bag_boston.fit(X_train, y_train)
```

Out[24]: RandomForestRegressor(max_features=12, random_state=0)

Argument **max_features** chỉ rằng all 12 predictors được xét tại each split—tức perform bagging. Performance trên test set:

```
In [25]: ax = subplots(figsize=(8,8))[1]
y_hat_bag = bag_boston.predict(X_test)
ax.scatter(y_hat_bag, y_test)
np.mean((y_test - y_hat_bag)**2)
```

Out[25]: 14.63

Test-set MSE của bagged regression tree là 14.63, khoảng half value từ optimally-pruned single tree. Có thể đổi number of trees từ default 100 bằng **n_estimators**.

In [26]:

```
bag_boston = RF(max_features=X_train.shape[1],
                 n_estimators=500,
                 random_state=0).fit(X_train, y_train)
y_hat_bag = bag_boston.predict(X_test)
np.mean((y_test - y_hat_bag)**2)
```

Out[26]: 14.61

Không có much change. Bagging và random forests không overfit do increasing number of trees, nhưng có thể underfit nếu number quá small. Random forest grow tương tự, chỉ dùng smaller `max_features`. By default, `RandomForestRegressor()` dùng p variables khi build regression forest (bagging), còn `RandomForestClassifier()` dùng $\sqrt{p}$ variables cho classification trees. Ở đây dùng `max_features=6`.

In [27]:

```
RF_boston = RF(max_features=6,
                random_state=0).fit(X_train, y_train)
y_hat_RF = RF_boston.predict(X_test)
np.mean((y_test - y_hat_RF)**2)
```

Out[27]: 20.04

Test-set MSE là 20.04; random forests hơi worse hơn bagging trong case này. Extract `feature_importances_` để xem importance của mỗi variable.

In [28]:

```
feature_imp = pd.DataFrame(
    {'importance': RF_boston.feature_importances_},
    index=feature_names)
feature_imp.sort_values(by='importance', ascending=False)
```

Out[28]:

	importance
lstat	0.368683
rm	0.333842
ptratio	0.057306
indus	0.053303
crim	0.052426
dis	0.042493
nox	0.034410
age	0.024327
tax	0.022368
rad	0.005048
zn	0.003238
chas	0.002557

Đây là relative measure của total decrease in node impurity từ splits trên variable đó, averaged over all trees (Figure 8.9 dùng measure này cho model trên **Heart**). Results cho thấy wealth level của community (`lstat`) và house size (`rm`) là by far two most important variables.

8.3.4 Boosting

Ta dùng `GradientBoostingRegressor()` từ `sklearn.ensemble` để fit boosted regression trees cho data Boston. Classification sẽ dùng `GradientBoostingClassifier()`. Argument `n_estimators=5000` nghĩa là 5000 trees; `max_depth=3` limits depth mỗi tree. `learning_rate` là λ

trong boosting description.

gradient boosting

In [29]:

```
boost_boston = GBR(n_estimators=5000,
                    learning_rate=0.001,
                    max_depth=3,
                    random_state=0)
boost_boston.fit(X_train, y_train)
```

Training error decreases có thể xem qua attribute `train_score_`. Để xem test error decreases, dùng `staged_predict()` lấy predicted values along path.

In [30]:

```
test_error = np.zeros_like(boost_boston.train_score_)
for idx, y_ in enumerate(boost_boston.staged_predict(X_test)):
    test_error[idx] = np.mean((y_test - y_)**2)

plot_idx = np.arange(boost_boston.train_score_.shape[0])
ax = subplots(figsize=(8,8))[1]
ax.plot(plot_idx,
        boost_boston.train_score_,
        'b',
        label='Training')
ax.plot(plot_idx,
        test_error,
        'r',
        label='Test')
ax.legend();
```

Dùng boosted model để predict `medv` trên test set:

In [31]:

```
y_hat_boost = boost_boston.predict(X_test);
np.mean((y_test - y_hat_boost)**2)
```

Out[31]: 14.48

Test MSE 14.48, similar bagging. Có thể boosting với shrinkage parameter λ khác trong (8.10). Source dùng default value 0.001 ở trên và sau đó lấy $\lambda = 0.2$.

In [32]:

```
boost_boston = GBR(n_estimators=5000,
                    learning_rate=0.2,
                    max_depth=3,
                    random_state=0)
boost_boston.fit(X_train,
                 y_train)
y_hat_boost = boost_boston.predict(X_test);
np.mean((y_test - y_hat_boost)**2)
```

Out[32]: 14.50

Trong case này, $\lambda = 0.2$ dẫn tới almost same test MSE như $\lambda = 0.001$.

8.3.5 Bayesian Additive Regression Trees

Section này demonstrate Python implementation của BART trong package `ISLP.bart`, fit model cho Boston housing data. Estimator `BART()` designed cho quantitative outcomes, dù có implementations khác cho logistic và probit models với categorical outcomes.

BART()

```
In [33]: bart_boston = BART(random_state=0, burnin=5, ndraw=15)
        bart_boston.fit(X_train, y_train)
```

```
Out[33]: BART(burnin=5, ndraw=15, random_state=0)
```

Với split test/training này, test error của BART similar random forest.

```
In [34]: yhat_test = bart_boston.predict(X_test.astype(np.float32))
        np.mean((y_test - yhat_test)**2)
```

```
Out[34]: 20.92
```

Ta check số lần mỗi variable xuất hiện trong collection of trees, summary tương tự variable importance plot cho boosting và random forests.

```
In [35]: var_inclusion = pd.Series(bart_boston.variable_inclusion_.mean(0),
                                index=D.columns)
        var_inclusion
```

```
Out[35]: crim      25.333333
        zn        27.000000
        indus     21.266667
        chas      20.466667
        nox       25.400000
        rm        32.400000
        age       26.133333
        dis       25.666667
        rad       24.666667
        tax       23.933333
        ptratio   25.000000
        lstat     31.866667
        dtype: float64
```

8.4 Exercises

Conceptual

1. Vẽ một example do bạn tự nghĩ ra về partition của two-dimensional feature space có thể tạo bởi recursive binary splitting. Example phải có ít nhất six regions. Vẽ decision tree tương ứng với partition này. Label đầy đủ mọi aspects của figures, gồm regions $R_1, R_2, \dots$, cutpoints $t_1, t_2, \dots$, v.v.
Hint: Result nên trông tương tự Figures 8.1 và 8.2.
2. Section 8.2.3 nói rằng boosting dùng depth-one trees (hay stumps) dẫn tới additive model, tức model dạng

$$f(X) = \sum_{j=1}^p f_j(X_j).$$

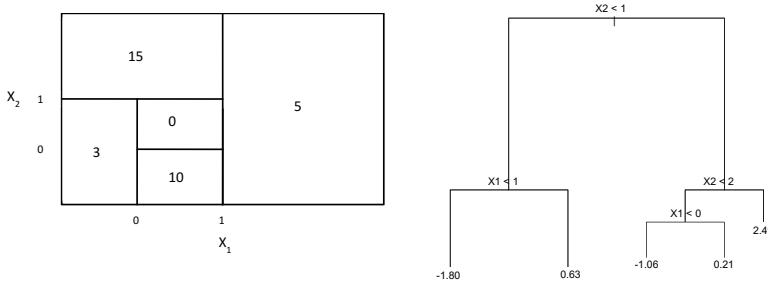
Explain why. Có thể bắt đầu từ (8.12) trong Algorithm 8.2.

3. Xét Gini index, classification error và entropy trong simple classification setting với two classes. Tạo một single plot display mỗi quantity

theo $\hat{p}_{m1}$. Trục x display $\hat{p}_{m1}$ từ 0 đến 1; trục y display Gini index, classification error, và entropy.

Hint: Với two classes, $\hat{p}_{m1} = 1 - \hat{p}_{m2}$. Có thể draw by hand, nhưng source lưu ý sẽ dễ hơn nhiều nếu làm trong R.

4. Câu hỏi này liên quan tới plots trong Figure 8.14.



HÌNH 8.14. Left: Partition của predictor space tương ứng Exercise 4(a). Right: Tree tương ứng Exercise 4(b).

- Sketch tree tương ứng partition của predictor space ở left-hand panel của Figure 8.14. Các numbers trong boxes chỉ mean của Y trong mỗi region.
 - Tạo diagram tương tự left-hand panel của Figure 8.14, dùng tree ở right-hand panel của same figure. Divide predictor space thành correct regions và indicate mean cho mỗi region.
5. Giả sử tạo ten bootstrapped samples từ data set có red và green classes. Apply classification tree cho mỗi bootstrap sample và, với một specific value của X , thu được 10 estimates của $P(\text{Class is Red} | X)$:

0.1, 0.15, 0.2, 0.2, 0.55, 0.6, 0.6, 0.65, 0.7, 0.75.

Có two common ways combine results thành single class prediction: majority vote approach và classify dựa trên average probability. Trong example này, final classification dưới mỗi approach là gì?

6. Provide detailed explanation của algorithm dùng để fit regression tree.

Applied

- Trong Section 8.3.3, random forests được apply cho data **Boston** với `max_features=6`, `n_estimators=100` và `n_estimators=500`. Tạo plot display test error từ random forests trên data set này với comprehensive range hơn của `max_features` và `n_estimators`. Có thể model plot theo Figure 8.10. Describe results.
- Trong lab, classification tree được apply lên data **Carseats** sau khi convert **Sales** thành qualitative response. Bây giờ predict **Sales** bằng regression trees và related approaches, treating response là quantitative variable.
 - Split data set thành training set và test set.

- (b) Fit regression tree trên training set. Plot tree và interpret results. Test MSE là bao nhiêu?
 - (c) Dùng cross-validation để determine optimal level của tree complexity. Pruning có improve test MSE không?
 - (d) Dùng bagging để analyze data. Test MSE là bao nhiêu? Dùng `feature_importance_` values để determine variables important nhất.
 - (e) Dùng random forests. Test MSE là bao nhiêu? Dùng `feature_importance_` values để determine variables important nhất. Describe effect của m , number of variables considered at each split, lên error rate.
 - (f) Analyze data bằng BART và report results.
9. Problem này dùng data set `OJ`, thuộc package `ISLP`.
- (a) Create training set gồm random sample 800 observations và test set gồm remaining observations.
 - (b) Fit tree trên training data, response là `Purchase`, predictors là other variables. Training error rate là bao nhiêu?
 - (c) Plot tree và interpret results. Tree có bao nhiêu terminal nodes?
 - (d) Dùng function `export_tree()` để produce text summary của fitted tree. Chọn một terminal node và interpret displayed information.
 - (e) Predict response trên test data và produce confusion matrix comparing test labels với predicted labels. Test error rate là bao nhiêu?
 - (f) Dùng cross-validation trên training set để determine optimal tree size.
 - (g) Produce plot với tree size trên x-axis và cross-validated classification error rate trên y-axis.
 - (h) Tree size nào ứng với lowest cross-validated classification error rate?
 - (i) Produce pruned tree tương ứng optimal tree size từ cross-validation. Nếu cross-validation không select pruned tree, tạo pruned tree với five terminal nodes.
 - (j) Compare training error rates giữa pruned và unpruned trees. Cái nào higher?
 - (k) Compare test error rates giữa pruned và unpruned trees. Cái nào higher?
10. Ta dùng boosting để predict `Salary` trong data set `Hitters`.
- (a) Remove observations thiếu salary information, rồi log-transform salaries.
 - (b) Create training set gồm first 200 observations và test set gồm remaining observations.
 - (c) Perform boosting trên training set với 1,000 trees cho một range

- values của shrinkage parameter λ . Produce plot với shrinkage values trên x-axis và corresponding training-set MSE trên y-axis.
- (d) Produce plot với shrinkage values trên x-axis và corresponding test-set MSE trên y-axis.
 - (e) Compare test MSE của boosting với test MSE từ two regression approaches ở Chapters 3 và 6.
 - (f) Variables nào appear to be most important predictors trong boosted model?
 - (g) Apply bagging lên training set. Test-set MSE là bao nhiêu?
11. Câu này dùng data set **Caravan**.
- (a) Create training set gồm first 1,000 observations và test set gồm remaining observations.
 - (b) Fit boosting model trên training set với **Purchase** là response và other variables là predictors. Dùng 1,000 trees và shrinkage value 0.01. Predictors nào appear to be most important?
 - (c) Dùng boosting model predict response trên test data. Predict rằng một person sẽ make a purchase nếu estimated probability of purchase $>20\%$. Form confusion matrix. Fraction của people predicted to make a purchase mà thực tế make one là bao nhiêu? Compare với results từ KNN hoặc logistic regression trên data set này.
12. Apply boosting, bagging, random forests, và BART lên data set tự chọn. Fit models trên training set và evaluate performance trên test set. Results accurate thế nào so với simple methods như linear hoặc logistic regression? Approach nào yield best performance?

9

Support Vector Machines

Chapter này discuss support vector machine (SVM), một classification approach developed trong computer science community vào 1990s và ngày càng popular. SVMs perform well trong nhiều settings và thường được xem là một trong những best “out of the box” classifiers.

Support vector machine là generalization của simple, intuitive classifier gọi là maximal margin classifier, giới thiệu ở Section 9.1. Dù elegant và simple, classifier này không apply được cho most data sets vì yêu cầu classes separable bởi linear boundary. Section 9.2 giới thiệu support vector classifier, extension của maximal margin classifier apply được rộng hơn. Section 9.3 giới thiệu support vector machine, further extension để accommodate non-linear class boundaries. SVMs intended cho binary classification với two classes; Section 9.4 discuss extensions khi có more than two classes. Section 9.5 discuss close connections giữa support vector machines và statistical methods khác như logistic regression.

People often loosely gọi maximal margin classifier, support vector classifier, và support vector machine đều là “support vector machines”. Để tránh confusion, chapter này distinguish carefully ba notions đó.

9.1 Maximal Margin Classifier

Trong section này, ta define hyperplane và introduce concept của optimal separating hyperplane.

9.1.1 What Is a Hyperplane?

Trong p -dimensional space, một *hyperplane* là flat affine subspace có dimension $p - 1$.¹ Ví dụ trong two dimensions, hyperplane là flat one-dimensional subspace—một line. Trong three dimensions, hyperplane là flat two-dimensional subspace—một plane. Với $p > 3$, khó visualize những notion của $(p - 1)$ -dimensional flat subspace vẫn áp dụng.

Mathematical definition khá simple. Trong two dimensions, hyperplane được define bởi

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 = 0, \quad (9.1)$$

¹Từ *affine* chỉ rằng subspace không nhất thiết đi qua origin.

cho parameters $\beta_0, \beta_1, \beta_2$. Nói (9.1) “defines” hyperplane nghĩa là mọi $X = (X_1, X_2)^T$ satisfying (9.1) là point on hyperplane. Đây đơn giản là equation of a line.

Extend tới p dimensions:

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p = 0. \quad (9.2)$$

Nếu $X = (X_1, X_2, \dots, X_p)^T$ trong p -dimensional space satisfies (9.2), thì X nằm trên hyperplane.

Nếu X không satisfy (9.2) mà

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p > 0, \quad (9.3)$$

thì X nằm về một side của hyperplane. Nếu

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p < 0, \quad (9.4)$$

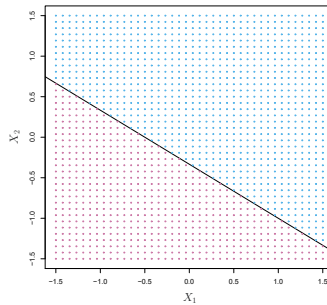
thì X ở side còn lại. Hyperplane divide p -dimensional space thành two halves; side của point được determine bằng sign của left-hand side (9.2). Figure 9.1 show two-dimensional example.

9.1.2 Classification Using a Separating Hyperplane

Giả sử có $n \times p$ data matrix X gồm n training observations trong p -dimensional space,

$$x_1 = \begin{pmatrix} x_{11} \\ \vdots \\ x_{1p} \end{pmatrix}, \dots, x_n = \begin{pmatrix} x_{n1} \\ \vdots \\ x_{np} \end{pmatrix}, \quad (9.5)$$

và observations thuộc two classes: $y_1, \dots, y_n \in \{-1, 1\}$, với -1 là một class và 1 class còn lại. Ta cũng có test observation $x^* = (x_1^*, \dots, x_p^*)^T$. Goal là develop classifier từ training data để classify test observation bằng feature measurements. Ta đã thấy LDA và logistic regression trong Chapter 4, classification trees, bagging, boosting trong Chapter 8; bây giờ xét approach dựa trên separating hyperplane.



HÌNH 9.1. Hyperplane $1 + 2X_1 + 3X_2 = 0$. Blue region là set of points với $1 + 2X_1 + 3X_2 > 0$, purple region là set of points với $1 + 2X_1 + 3X_2 < 0$.

Giả sử có thể construct hyperplane perfectly separates training observations theo class labels. Left-hand panel của Figure 9.2 show ba such

separating hyperplanes. Label blue class $y_i = 1$, purple class $y_i = -1$. Separating hyperplane có property

$$\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip} > 0 \quad \text{if } y_i = 1, \quad (9.6)$$

và

$$\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip} < 0 \quad \text{if } y_i = -1. \quad (9.7)$$

Equivalently,

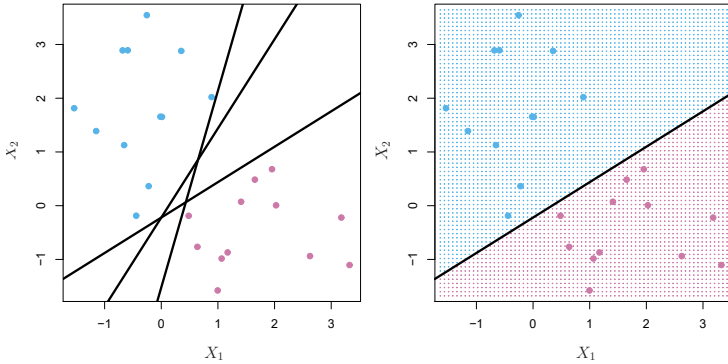
$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip}) > 0 \quad (9.8)$$

cho mọi $i = 1, \dots, n$.

Nếu separating hyperplane exists, natural classifier là assign test observation theo side của hyperplane. Right-hand panel Figure 9.2 minh họa. Ta classify x^* theo sign của

$$f(x^*) = \beta_0 + \beta_1 x_1^* + \beta_2 x_2^* + \cdots + \beta_p x_p^*.$$

Nếu positive assign class 1; nếu negative assign class -1 . Magnitude của $f(x^*)$ cũng useful: far from zero nghĩa là x^* far from hyperplane và confidence cao hơn; near zero nghĩa là observation near hyperplane và class assignment less certain.



HÌNH 9.2. Left: Two classes of observations, blue và purple, mỗi observation measured trên two variables. Ba separating hyperplanes trong nhiều possibilities được show bằng black. Right: Một separating hyperplane in black. Blue/purple grid chỉ decision rule: test observation trong blue region assign blue class, trong purple region assign purple class.

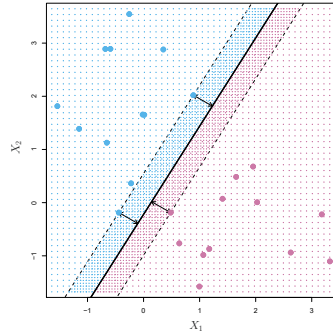
Không surprising, classifier dựa trên separating hyperplane dẫn tới linear decision boundary.

9.1.3 The Maximal Margin Classifier

Nếu data perfectly separable bởi hyperplane, thường tồn tại infinite number of separating hyperplanes. Một given hyperplane có thể shift slightly up/down hoặc rotate mà chưa touch observations. Left panel Figure 9.2 show three possibilities. Để build classifier, cần reasonable way chọn một hyperplane.

Natural choice là *maximal margin hyperplane*, còn gọi *optimal separating hyperplane*: separating hyperplane farthest from training observations. Với mỗi separating hyperplane, compute perpendicular distance từ từng training observation; smallest distance là minimal distance đến hyperplane, gọi là *margin*. Maximal margin hyperplane là one với largest margin. Classifier dựa trên side của maximal margin hyperplane gọi là *maximal margin classifier*. Ta hope large training margin cũng dẫn đến large test margin và correct classification; dù often successful, maximal margin classifier có thể overfit khi p large.

*maximal margin
hyperplane;
optimal separating
hyperplane;
margin; maximal
margin classifier*



HÌNH 9.3. Two classes shown blue và purple. Maximal margin hyperplane là solid line. Margin là distance từ solid line tới một trong hai dashed lines. Hai blue points và một purple point nằm trên dashed lines là support vectors; arrows chỉ distance từ các points này tới hyperplane. Purple/blue grid indicate classifier decision rule.

Nếu $\beta_0, \beta_1, \dots, \beta_p$ là coefficients của maximal margin hyperplane, classifier classifies x^* theo sign của $f(x^*) = \beta_0 + \beta_1 x_1^* + \dots + \beta_p x_p^*$.

Figure 9.3 show maximal margin hyperplane trên data Figure 9.2. So với right panel Figure 9.2, maximal margin hyperplane có greater minimal distance tới observations, tức larger margin. Có thể xem nó là mid-line của widest “slab” có thể insert giữa two classes.

Ba training observations trong Figure 9.3 equidistant từ maximal margin hyperplane và nằm trên dashed margin lines. Chúng gọi là *support vectors*: vectors trong p -dimensional space và “support” maximal margin hyperplane vì nếu move slightly thì hyperplane cũng move. Hyperplane depends directly on support vectors, không on other observations; movement của other observations không affect hyperplane miễn chúng không cross margin boundary. Property này sẽ reappear với support vector classifier và support vector machines.

support vector

9.1.4 Construction of the Maximal Margin Classifier

Với n training observations $x_1, \dots, x_n \in \mathbb{R}^p$ và class labels $y_1, \dots, y_n \in \{-1, 1\}$, maximal margin hyperplane là solution của optimization problem

$$\text{maximize}_{\beta_0, \beta_1, \dots, \beta_p, M} \quad M, \quad (9.9)$$

$$\text{subject to} \quad \sum_{j=1}^p \beta_j^2 = 1, \quad (9.10)$$

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip}) \geq M, \quad \forall i = 1, \dots, n. \quad (9.11)$$

Problem (9.9)–(9.11) simpler than it looks. Constraint (9.11) guarantee mỗi observation on correct side của hyperplane nếu $M > 0$. Để merely on correct side chỉ cần product > 0 ; (9.11) yêu cầu thêm cushion.

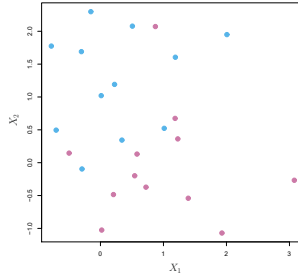
Constraint (9.10) không thật sự constrain hyperplane: nếu $\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip} = 0$ defines hyperplane thì $k(\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip}) = 0$ cũng defines same hyperplane cho any $k \neq 0$. Nhưng (9.10) gives meaning to (9.11): perpendicular distance từ i th observation tới hyperplane bằng

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip}).$$

Do đó (9.10), (9.11) ensure every observation on correct side và at least distance M from hyperplane. M chính là margin, và optimization chooses coefficients maximize M . Details giải optimization nằm ngoài scope.

9.1.5 The Non-separable Case

Maximal margin classifier là natural nếu separating hyperplane exists. Nhưng trong nhiều cases không có separating hyperplane, nên không có maximal margin classifier; optimization (9.9)–(9.11) không có solution với $M > 0$. Figure 9.4 show example. Ta không thể exactly separate classes, nhưng có thể extend separating hyperplane để almost separate classes bằng so-called soft margin. Generalization này gọi là support vector classifier.



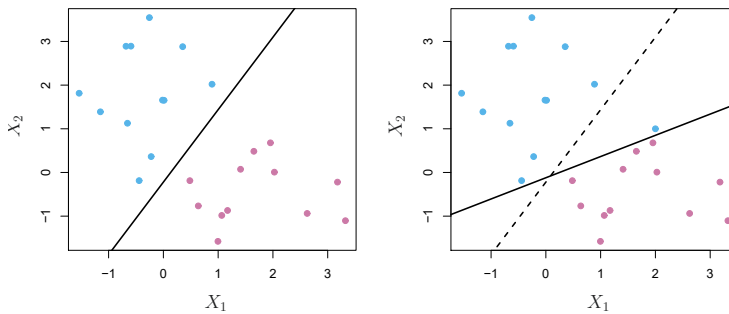
HÌNH 9.4. Two classes shown blue và purple. Classes không separable bởi hyperplane, nên maximal margin classifier không thể dùng.

9.2 Support Vector Classifiers

9.2.1 Overview of the Support Vector Classifier

Figure 9.4 cho thấy observations từ two classes không nhất thiết separable bởi hyperplane. Ngay cả khi separating hyperplane exists, classifier dựa trên perfect separation đôi khi không desirable: nó necessarily perfectly classify all training observations và vì thế sensitive to individual observations. Figure 9.5 show example: thêm single observation ở right panel làm maximal margin hyperplane change dramatically. New hyperplane có tiny margin; moreover high sensitivity to one observation suggests overfitting.

Trong case này, ta có thể prefer classifier dựa trên hyperplane không perfectly separate two classes để đổi lấy robustness với individual observations và better classification của most training observations. Nói cách khác, có thể worthwhile misclassify vài training observations để classify remaining observations tốt hơn.



HÌNH 9.5. Left: Two classes shown blue và purple cùng maximal margin hyperplane. Right: Add một blue observation, gây dramatic shift trong maximal margin hyperplane shown solid. Dashed line là maximal margin hyperplane trước khi thêm point.

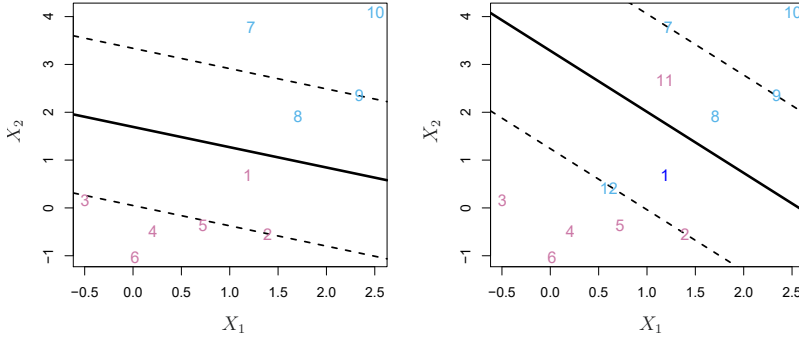
Support vector classifier, đôi khi gọi soft margin classifier, làm đúng điều này. Thay vì seek largest margin sao cho every observation vừa ở correct side của hyperplane vừa correct side của margin, ta allow some observations ở incorrect side của margin hoặc thậm chí incorrect side của hyperplane. Margin gọi là soft vì có thể bị violated bởi training observations. Left panel Figure 9.6: most observations correct side của margin, small subset wrong side.

Observation có thể không chỉ wrong side of margin mà còn wrong side of hyperplane. Khi không có separating hyperplane, điều này inevitable. Observations wrong side of hyperplane là training observations misclassified bởi support vector classifier. Right panel Figure 9.6 illustrate scenario.

support vector
classifier; soft
margin classifier

9.2.2 Details of the Support Vector Classifier

Support vector classifier classifies test observation theo side của hyperplane. Hyperplane được choose để correctly separate most training observations thành two classes nhưng may misclassify a few.



HÌNH 9.6. Left: Support vector classifier fit vào small data set. Hyperplane là solid line, margins là dashed lines. Purple observations: 3, 4, 5, 6 ở correct side of margin; 2 on margin; 1 wrong side of margin. Blue observations: 7, 10 correct side; 9 on margin; 8 wrong side of margin. Không observation nào wrong side of hyperplane. Right: Same panel với additional points 11 và 12; cả hai wrong side of hyperplane và margin.

Hyperplane là solution của optimization problem

$$\text{maximize}_{\beta_0, \beta_1, \dots, \beta_p, \varepsilon_1, \dots, \varepsilon_n, M} M, \quad (9.12)$$

$$\text{subject to} \quad \sum_{j=1}^p \beta_j^2 = 1, \quad (9.13)$$

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip}) \geq M(1 - \varepsilon_i), \quad (9.14)$$

$$\varepsilon_i \geq 0, \quad \sum_{i=1}^n \varepsilon_i \leq C. \quad (9.15)$$

Ở đây C là nonnegative tuning parameter. Như (9.11), M là width của margin, cần maximize. $\varepsilon_1, \dots, \varepsilon_n$ là *slack variables* cho phép individual observations nằm wrong side of margin hoặc hyperplane. Sau khi solve (9.12)–(9.15), classify test observation x^* như trước theo sign của $f(x^*) = \beta_0 + \beta_1 x_1^* + \dots + \beta_p x_p^*$.

slack variable

Insight vào problem đến từ slack variables. Nếu $\varepsilon_i = 0$, i th observation ở correct side of margin như Section 9.1.4. Nếu $\varepsilon_i > 0$, observation wrong side of margin và được nói là *violated the margin*. Nếu $\varepsilon_i > 1$, nó ở wrong side of hyperplane.

Tuning parameter C bounds sum của ε_i 's, nên determines number và severity của violations to margin (và hyperplane) tolerated. Có thể xem C là budget cho amount margin có thể violated bởi n observations. Nếu $C = 0$, không có violation budget, nên $\varepsilon_1 = \dots = \varepsilon_n = 0$ và (9.12)–(9.15) trở lại maximal margin optimization (9.9)–(9.11), nếu classes separable. Với $C > 0$, không quá C observations có thể wrong side of hyperplane vì mỗi such observation có $\varepsilon_i > 1$ và sum không vượt C .

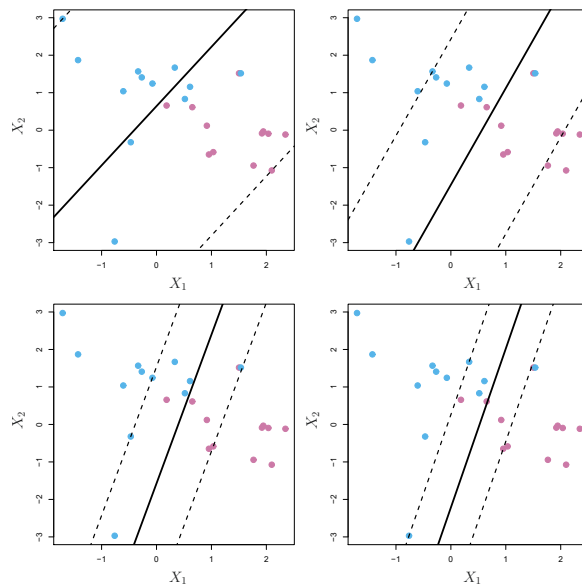
Khi budget C tăng, ta tolerate margin violations nhiều hơn nên margin widen. Khi C giảm, tolerance giảm và margin narrows. Figure 9.7 show example. Trong practice, C treated như tuning parameter, thường chọn bằng cross-validation. C controls bias-variance trade-off: small C seeks

narrow margins rarely violated, classifier fit data hard, potentially low bias/high variance; larger C gives wider margin with more violations, fitting less hard, potentially higher bias/lower variance.

Optimization (9.12)–(9.15) có interesting property: chỉ observations nằm on margin hoặc violate margin affect hyperplane và resulting classifier. Observation strictly on correct side of margin không affect classifier; move observation đó sẽ không change classifier miễn nó vẫn ở correct side. Observations directly on margin hoặc wrong side of margin gọi là support vectors, và chúng affect support vector classifier.

Điều này consistent với role của C trong bias-variance trade-off. Khi C large, margin wide, many observations violate margin, nên có many support vectors; nhiều observations determine hyperplane, classifier low variance nhưng potentially high bias. Top-left Figure 9.7 illustrate. Khi C small, fewer support vectors; classifier low bias/high variance. Bottom-right panel Figure 9.7 có only eight support vectors.

Decision rule chỉ dựa trên potentially small subset training observations—support vectors—làm classifier robust với observations far away from hyperplane.



HÌNH 9.7. Support vector classifier fit bằng four values khác nhau của tuning parameter C trong (9.12)–(9.15). Largest C ở top-left; smaller values ở top-right, bottom-left, bottom-right. Khi C large, tolerance cho observations wrong side of margin cao, nên margin large. Khi C decreases, tolerance decreases và margin narrows.

Property này khác một số classification methods ở preceding chapters, như linear discriminant analysis. LDA classification rule depends on mean của all observations within each class và within-class covariance matrix computed using all observations. Ngược lại, logistic regression, unlike LDA, có very low sensitivity với observations far from decision boundary. Section 9.5 sẽ cho thấy support vector classifier và logistic regression closely related.